Task1_环境配置报告

Python、Conda、VS Code 与 GPU 计算环境 | 基于当前环境及保存的代码复核

1 环境配置结果

本项目使用 VS Code 编写和运行 Python，独立 Conda 环境名为 comsen-answer。解释器与依赖放在 D 盘，避免不同 Python 环境混用。当前报告重新确认了实际 Python、主要包版本与 GPU 张量运算。

项目	实际记录
操作系统	Windows；PowerShell 终端
Python	3.11.17
解释器	D:\anaconda\envs\comsen-answer\python.exe
PyTorch	2.10.0+cu128
torchvision	0.25.0+cu128
PyTorch CUDA 运行时	12.8
GPU	NVIDIA GeForce RTX 5070 Laptop GPU
Conda	未能从当前系统复核；此前终端记录为 4.5.11

此前的 Python 3.13.5 是另一解释器的版本。项目实际运行的是上述 Python 3.11 环境；Conda 的版本号与环境中 Python 的版本号是不同概念。当前 Conda 命令未能自动复核，因此 4.5.11 仅作为此前用户终端记录保留。

2 创建与选择解释器

```
conda create -n comsen-answer python=3.11 -y
# VS Code: Python: Select Interpreter
D:/anaconda/envs/comsen-answer/python.exe
```

VS Code 中选择该解释器后，运行 hello.py，检查 sys.executable。终端未显示环境名前缀并不影响直接用解释器绝对路径执行；安装依赖也使用同一个解释器的 -m pip。上述创建命令为已完成步骤的记录，不要求重复创建环境。

3 基础运行与 GPU 验证

hello.py 输出 Hello Python、解释器路径和 Python 版本。task3/00_check_torch.py 检查 PyTorch 版本、CUDA 可用性及显卡名称，并在 GPU 上执行矩阵乘法。报告生成时使用相同算式复核通过。

```
A = [[1, 2], [3, 4]]
A @ A = [[7, 10], [15, 22]]
CUDA available: True
```

矩阵乘法检验不仅查看 GPU 是否存在，还确认张量能够放到 cuda 设备并完成计算。nvidia-smi 显示的 CUDA 版本是驱动支持能力，torch.version.cuda 是 PyTorch 构建使用的运行时版本，两者数值不必相同。

NVIDIA：CUDA 与驱动兼容性说明

功能	本项目使用的库 / 版本
图像读写、轮廓、追踪	opencv-contrib-python 5.0.0.93
图片显示与保存	Pillow 12.3.0
数组与表格	NumPy 2.4.6；pandas 3.0.6
结果绘图	Matplotlib 3.11.2

4 Python 基础与项目组织

变量保存数据，列表和字典组织样本与参数；if 判断读取是否成功，for/while 遍历批次与视频帧，函数封装处理流程。Python 使用缩进界定代码块；try/finally 用于在异常或退出时释放视频资源。Path(__file__).resolve().parent 让文件定位不依赖终端当前目录。

工作目录为 D:/comsen/comsen-answer。task2 保存图像处理和追踪；task3 保存函数拟合与 MNIST；task4 提交图像分类与时序预测，分目录组织。原始数据、源码和 outputs 分开存放，便于复核结果。

安装过程中的 Scripts 目录不在 PATH 提示不等于库安装失败；项目通过指定解释器导入模块。此前大体积 PyTorch 下载出现磁盘空间不足，随后已安装成功并完成 CUDA 验证。此处记录实际结果，不根据提示推断训练性能。